

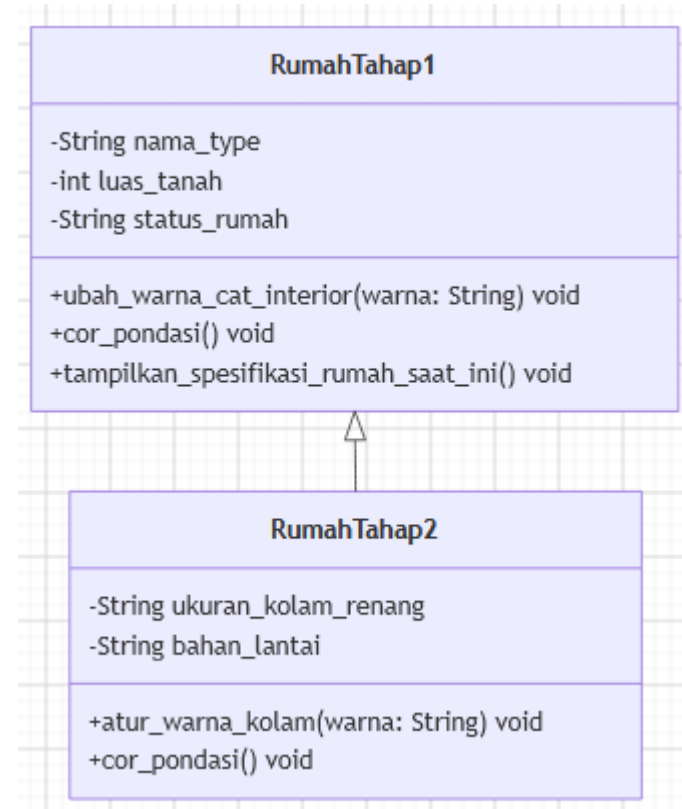
Class Diagram & Sequence Diagram

Materi

- Pada pertemuan 1, kita telah membahas tentang 4 pillar OOP yaitu:
 - Encapsulation
 - Abstraction
 - Inheritance
 - Polymorphims
- Pada pertemuan 2 kita akan membahas bagaimana konsep tersebut digambarkan dalam diagram?

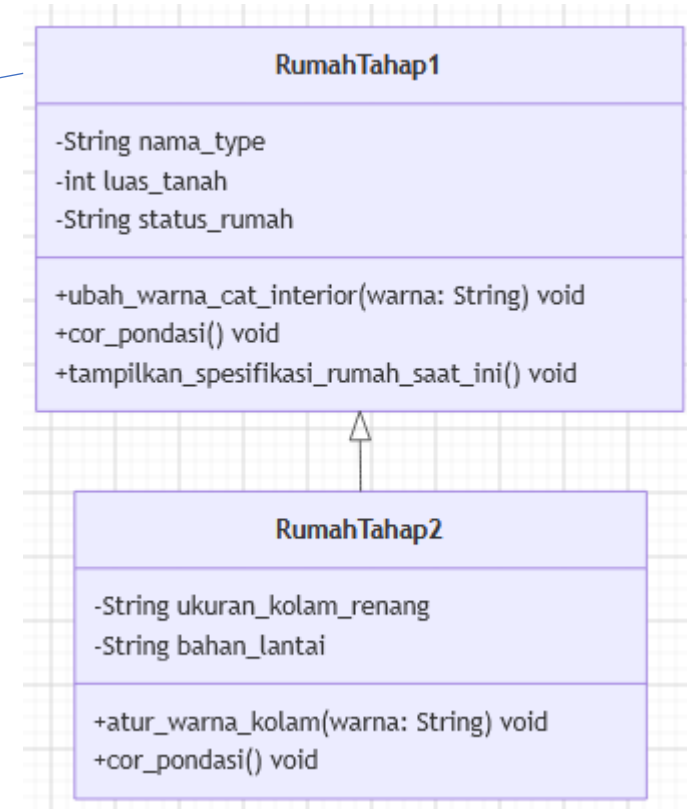
Class Diagram

- Class diagram adalah visualisasi blueprint dari Class, yang terdiri attribute (property), behaviour (metode) dan hubungan antara suatu class dengan class lainnya.



Anatomi Class Diagram

- Compartements
 - Top: Class Name
 - Middle: Attributes (variabel, tipe data)
 - Bottom: Operations (method, functions)
- Access Modifier:
 - + Public
 - - Private
 - # Protected

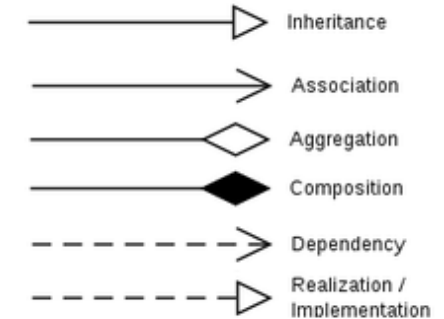


Nb.

- Semua variable ataupun function pada Python adalah bersifat +Public
- Developer disarankan untuk menggunakan konversi penamaan `_identifier` untuk semua Protected dan `__identifier` untuk semua private, sehingga developer lainnya paham bahwa identifier tersebut diperlakukan sebagai Public, Protected ataupun Private.

Class Relationship

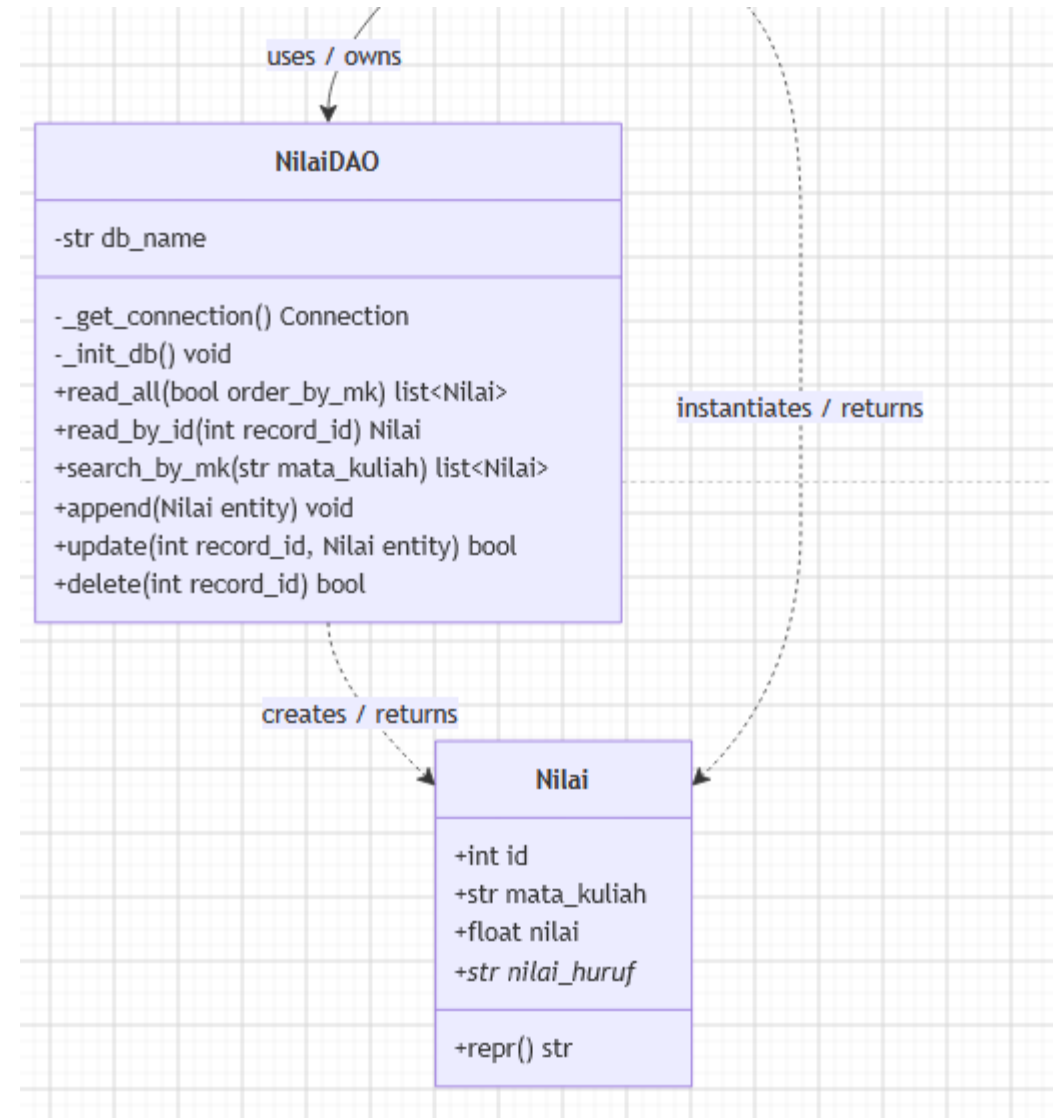
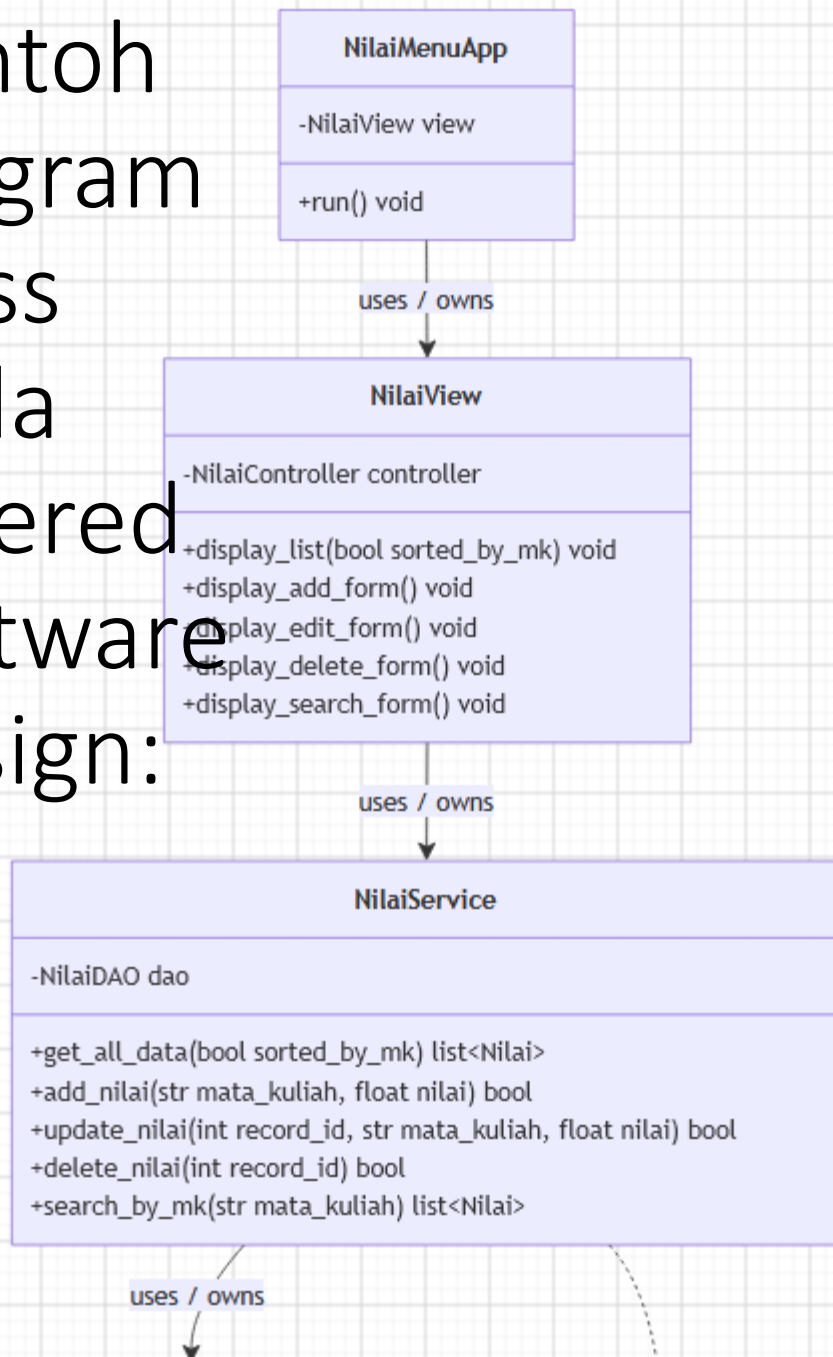
- Garis dan Panah yang menghubungkan Class menunjukkan interaksi diantara mereka:
 - Inheritance (Generalization): hubungan subclass dengan parent class (extends), digambarkan dengan garis solid dengan hollow triangle menunjuk ke parent class.
 - Association (Use/Own): koneksi dasar antara dua class yang mana yang satu menggunakan atau berinteraksi, digambarkan dengan solid line sederhana, dan ada dua bentuk khusus Association untuk yang hubungan lebih kuat.
 - Aggregation (has a): hubungan ketika child dapat bebas eksis tergantung kepada parent (misalkan Invoice has Customer), digambarkan dengan hollow diamond.
 - Composition (strong has a): hubungan dimana Ketika child tidak dapat ada tanpa parent (Invoice_Header has Invoice_Details), digambarkan dengan filled diamond.
 - Dependency (instantiate/return): Suatu hubungan pemakaian sementara dimana suatu class tergantung kepada class lainnya untuk berfungsi (misalkan class koneksi database, class authentication). Digambarkan dengan dashed arrow.



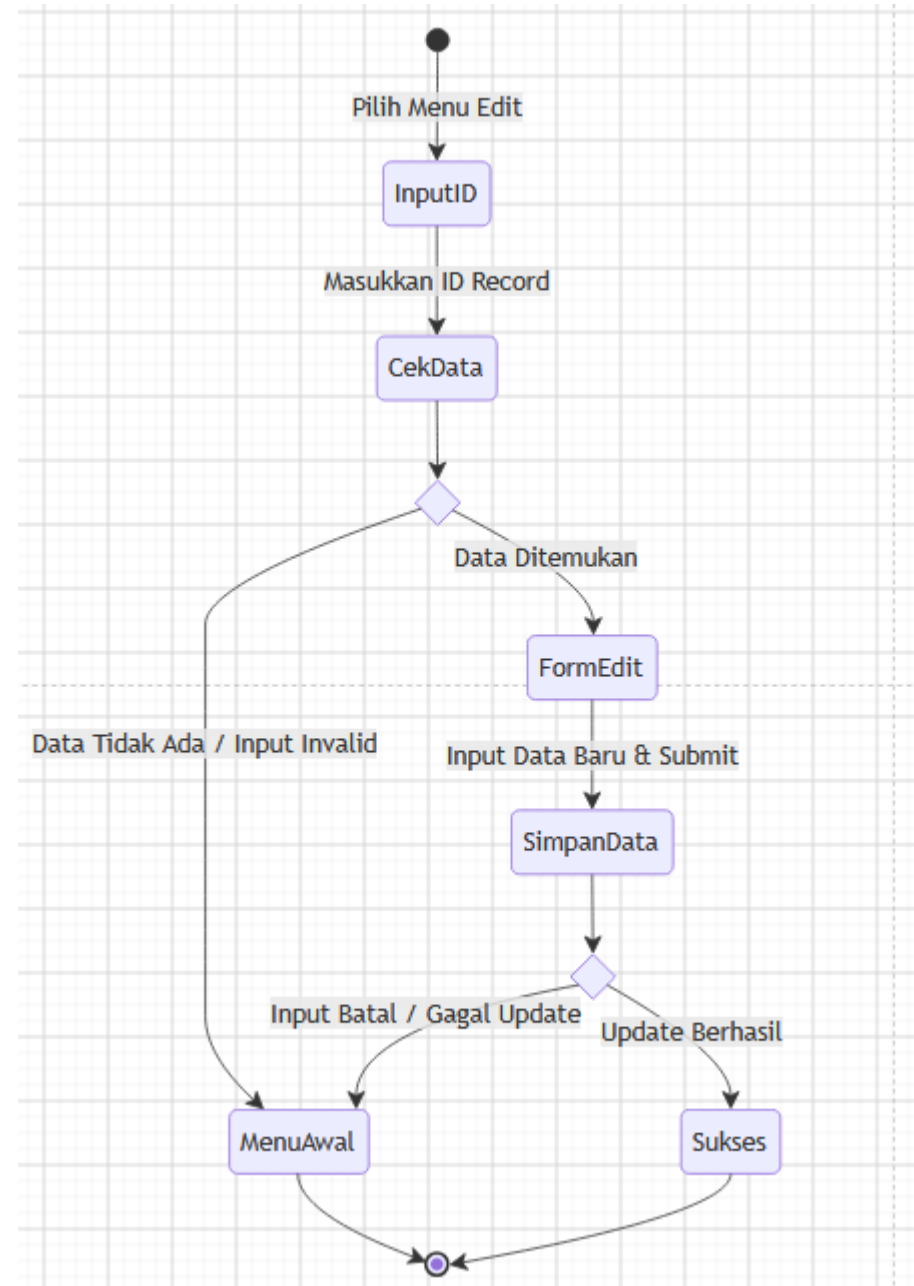
- Customer bisa didaftarkan walaupun belum ada Invoice atas Customer tersebut
- Invoice_Details tidak bisa ada tanpa Invoice_Header

- Implementation digunakan pada Interface pada OOP

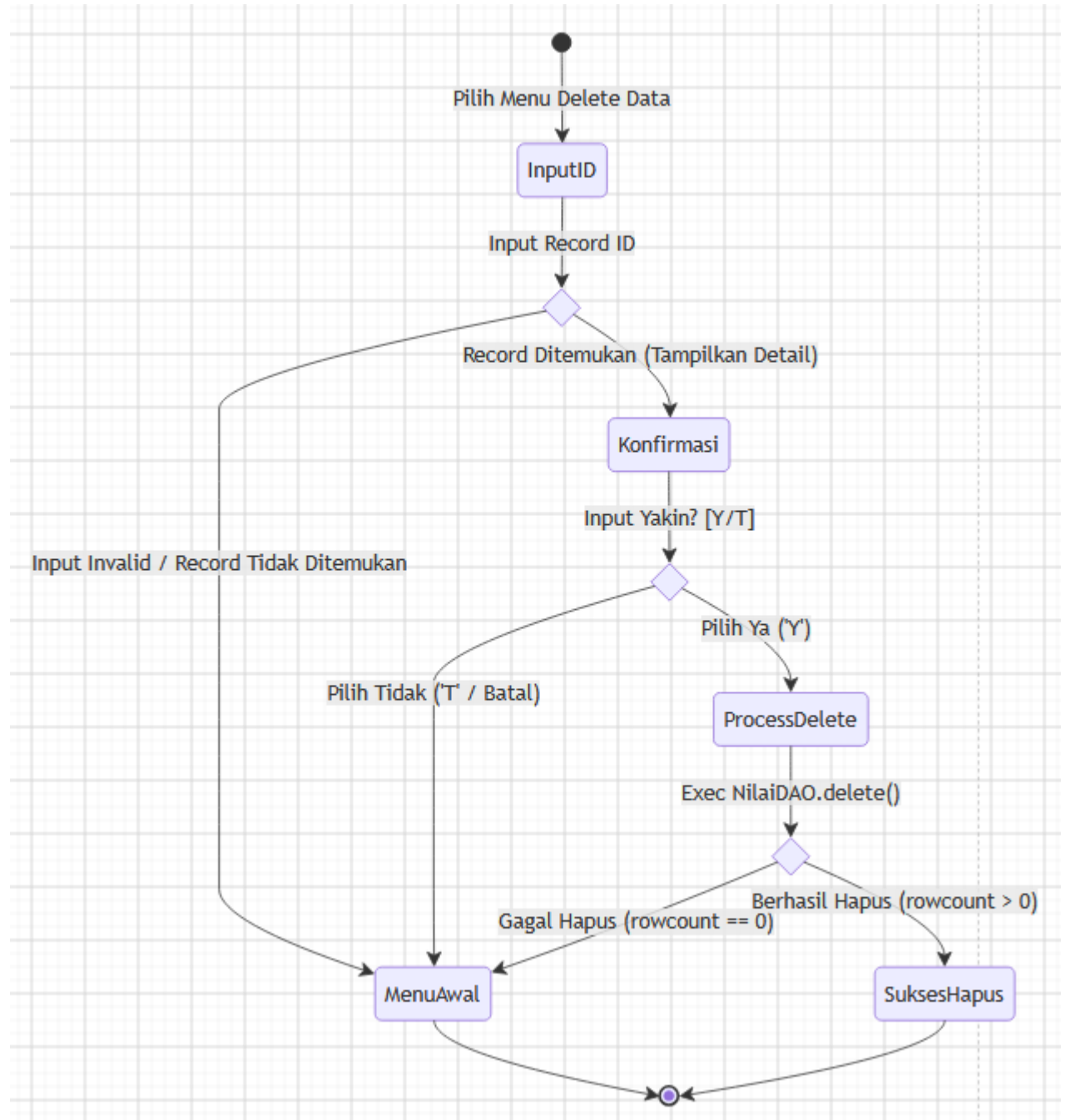
Contoh Diagram Class pada Layered Software Design:



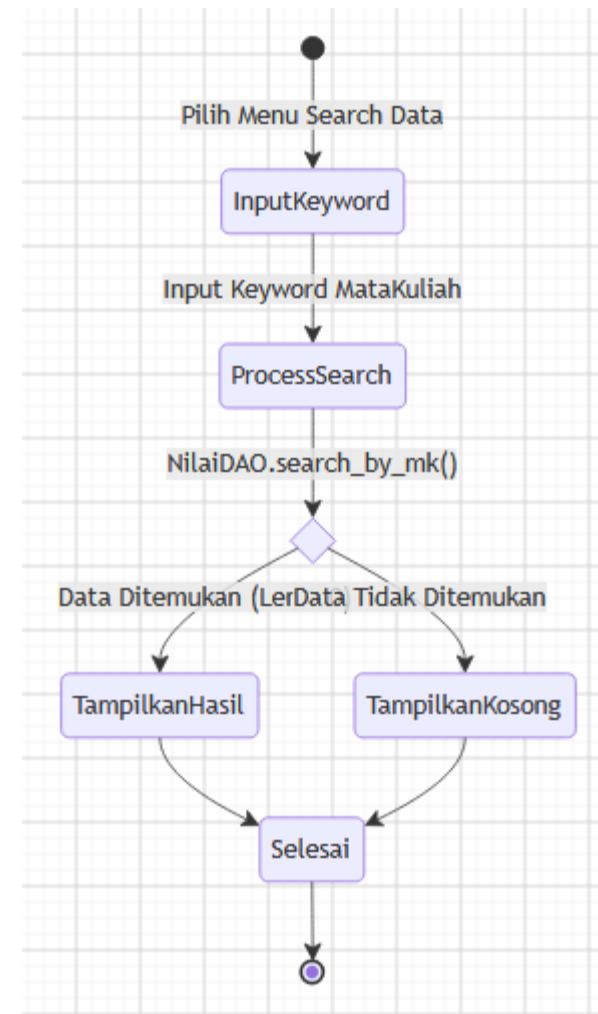
Activity Diagram UC-03 Edit Data



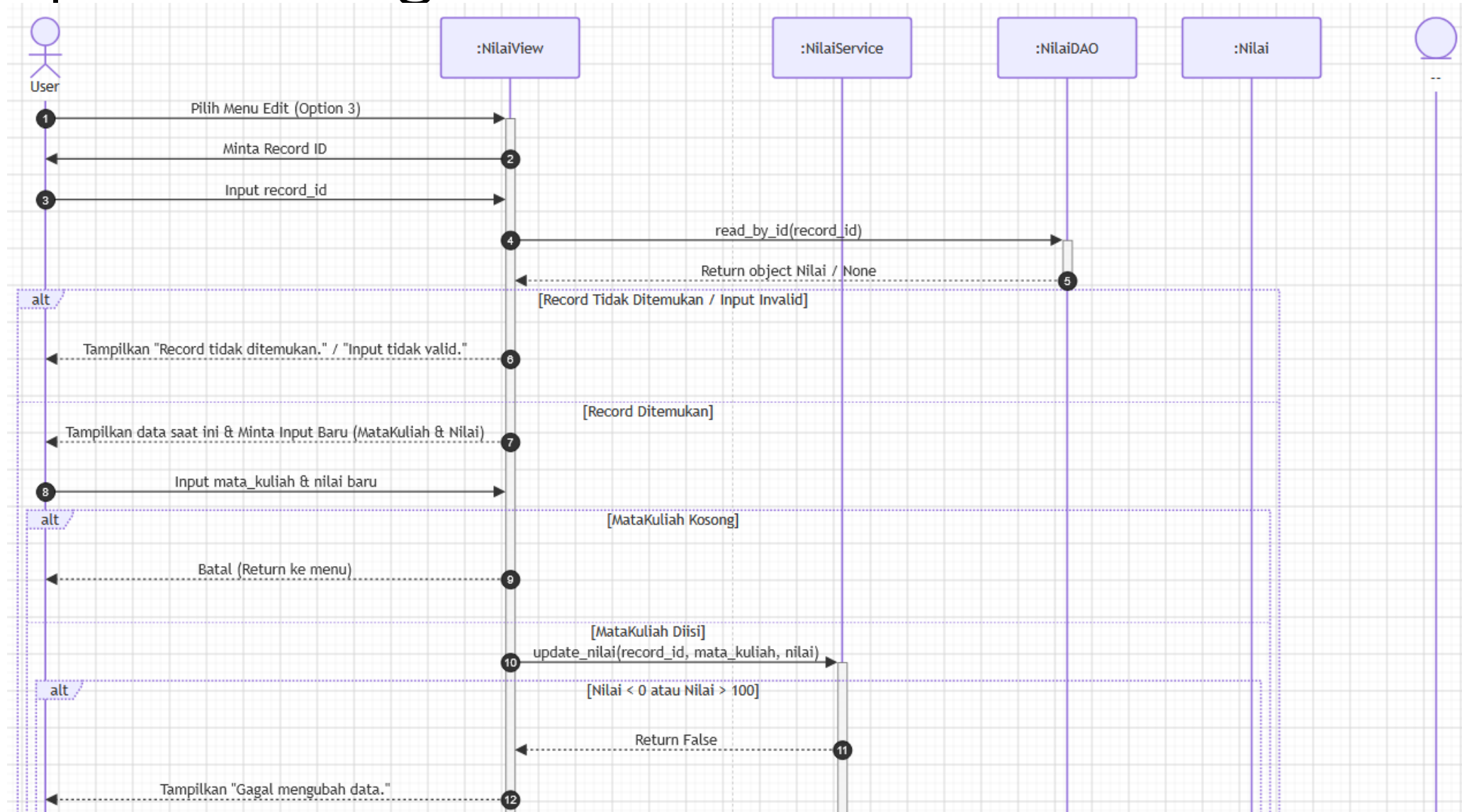
Activity Diagram UC-04 Hapus Data



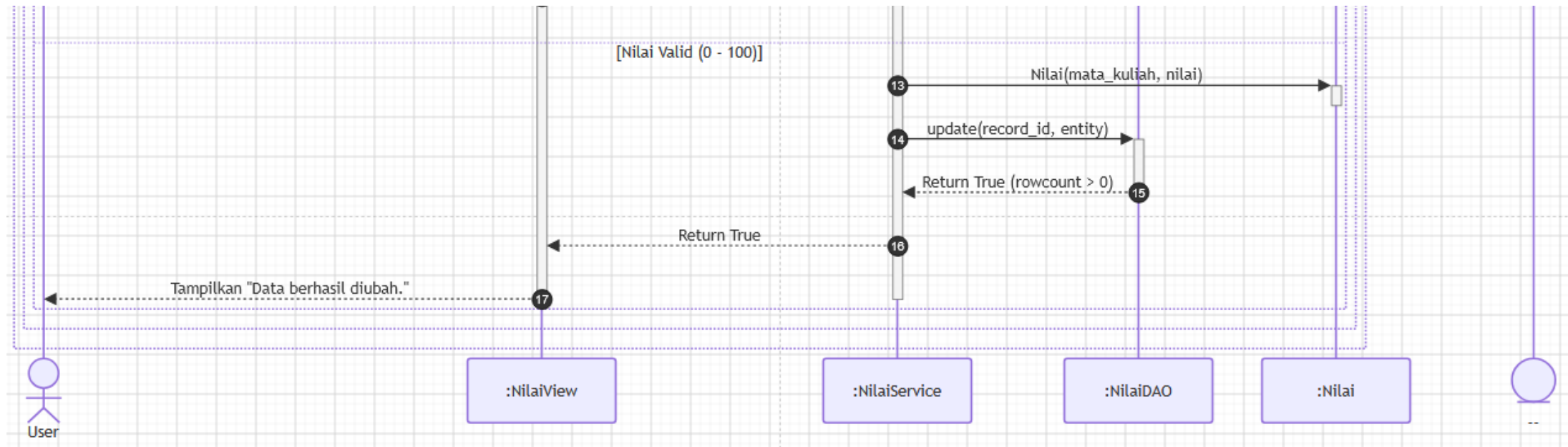
Activity Diagram UC-05 Cari Data



Sequence Diagram



Sambungan



SQLite

- SQLite adalah database engine ringan dan serverless yang embedded langsung pada aplikasi daripada berjalan pada server process yang terpisah.
- Berbeda dengan database seperti MySQL ataupun PostgreSQL yang membutuhkan arsitektur client-server, SQLite menyimpan keseluruhan struktur database pada satu file tunggal pada media penyimpanan.

SQLite pada Python

- Pemakaian SQLite didalam kode Python dapat dilakukan secara langsung karena module sqlite3 adalah dibangun langsung kedalam Python standard library.
- Anda tidak perlu menginstall secara terpisah dengan menggunakan pip.

Create Table

```
import sqlite3

# 1. CONNECT & CREATE TABLE
# Opens 'academic.db' and automatically saves (commits) the new table.
with sqlite3.connect("academic.db") as conn:
    cursor = conn.cursor()

    # Create the 'nilai' table using your exact schema
    cursor.execute("""
        CREATE TABLE IF NOT EXISTS nilai (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            mata_kuliah TEXT NOT NULL,
            nilai REAL NOT NULL
        )
    """)
    print("Table 'nilai' created successfully.")
```

Insert

```
# 2. INSERT (Add Data)
with sqlite3.connect("academic.db") as conn:
    cursor = conn.cursor()

    # Insert Subject 1
    cursor.execute(
        "INSERT INTO nilai (mata_kuliah, nilai) VALUES (?, ?)",
        ("Basis Data", 85.5),
    )

    # Insert Subject 2
    cursor.execute(
        "INSERT INTO nilai (mata_kuliah, nilai) VALUES (?, ?)",
        ("Pemrograman Python", 78.0),
    )

    # Insert Subject 3
    cursor.execute(
        "INSERT INTO nilai (mata_kuliah, nilai) VALUES (?, ?)",
        ("Matematika Diskrit", 92.3),
    )
    print("Three subjects added to database.")
```

Select

```
# 3. SELECT (Read Data BEFORE changes)
with sqlite3.connect("academic.db") as conn:
    # Use row_factory to access results by column names (e.g., row['mata_kuliah'])
    conn.row_factory = sqlite3.Row
    cursor = conn.cursor()

    cursor.execute("SELECT * FROM nilai")
    all_rows = cursor.fetchall()

    print("\nGrades BEFORE Updates:")
    for row in all_rows:
        print(
            f"ID: {row['id']} | Subject: {row['mata_kuliah']} | Grade: {row['nilai']}"
        )
```

Update

```
# 4. UPDATE (Modify Data)
with sqlite3.connect("academic.db") as conn:
    cursor = conn.cursor()

    # Change the grade of 'Basis Data' (ID: 1) from 85.5 to 90.0
    cursor.execute(
        "UPDATE nilai SET nilai = ? WHERE id = ?",
        (90.0, 1),
    )
    print("\nUpdated ID 1 (Basis Data) grade to 90.0.")
```

Delete

```
# 5. DELETE (Remove Data)
with sqlite3.connect("academic.db") as conn:
    cursor = conn.cursor()

    # Delete 'Pemrograman Python' (ID: 2) entirely from records
    cursor.execute(
        "DELETE FROM nilai WHERE id = ?",
        (2,),
    )
    print("Deleted ID 2 (Pemrograman Python) from records.")
```

Tugas

- Buka file oop2.py, jalankan dan lakukan refleksi
 - Bandingkan antara oop2.py dengan oop1.py
 - Bagaimana penyimpanan data pada masing-masing aplikasi
 - Lapisan manakah dari aplikasi yang banyak mengalami perubahan?
 - Mengapa?
 - Jelaskan manfaat pengembangan dengan Layered Architecture.
 - Jelaskan kelemahan dari pengembangan dengan Layerd Architecture.
- Gambarkan Class Diagram:
 - NilaiMenuApp
 - NilaiView
 - NilaiController
 - NilaiDAO
 - Nilai
- Gambarkan Activity Diagram
 - Perbaiki Data
 - Hapus Data
 - Cari Dta
- Gambarkan Sequence Diagram
 - Perbaiki Data

Tugas OBL Mid-Test (Langkah 0)

- Bentuklah tim yang masing-masing terdiri dari 2 orang